Chapter 2: Basic Static Techniques

Static Analysis miêu tả quá trình phân tích code và cấu tạo của một chương trình để hiểu được tác dụng của nó. (Chương trình hiện tại k đc chạy)

Chương 2 nói về nhiều cách đề trích xuất thông tin hữu dụng từ các file exes:

- Sử dụng tool antivirus để dám chực sự độc hại của file
- Sử dụng hash để xác định malware
- Thu thập thông tin từ string, functions, headers của một file

Tuỳ vào từng cách sẽ có các thông tin khác nhau và mình sẽ thường sửu dụng nhiều cách để có nhiều thông tin nhất có thể.

Antivirus Scanning

- Cách đầu tiên tốt nhất là chạy file đó qua các nhiều trình antivirus bởi vì chúng sẽ xác định được luôn. Tuy nhiên antivirus chỉ phụ thuộc vào databases của các mảnh suspicious code (file signatures), cũng như là behavioral, pattern-matching analysis (heuristic) để xác định file.
- Do đó thường thì malware có thể không bị phát hiện ra (do author có thể thay đổi code/ code của nó thậm chí không nằm trong database) . Các malware mới thường sẽ có thể bypass các cách phát hiện ấy.
- Các antirivus khác nhau có databases, signatures, heuristics kahcs nhau nên nên chạy nhiều loại antivirus. (Web virustotal)

Hashing: A Fingerprint for Malware

- Malicious software được chạy qua chương trình hashing mà tạo ra một unique hash giúp xác định malware (một kiểu fingerprint).
- Hash MD5 được sử dụng phổ biến nhất cho phân tích malware, SHA-1 cũng phổ biến.

- VD sử dụng md5deep cho Solitaire

> For example, using the freely available md5deep program to calculate the hash of the Solitaire program that comes with Windows would generate the following output:
>
> ```
> C:\>md5deep c:\WINDOWS\system32\sol.exe
> 373e7a863a1a345c60edb9e20ec3231  c:\WINDOWS\system32\sol.exe
> ```
>
> The hash is **373e7a863a1a345c60edb9e20ec3231**.

→ Sau khi mà mình đã có cái unique hash của malware mình có thể:

- Sử dụng làm một label
- Chia sẻ với các analyst khác để giúp họ xác định malware
- Tra file hash đó lên mạng xem file đó đã được xác định hay chưa.

Finding Strings

- Một chương trình chứa strings nếu nó in ra một message, kết nối tới một URL, hoặc copy một file tới một location cụ thể.
- Việc tìm qua Strings của một file có thể chỉ ra cách hoạt động của file ấy.
- Có thể sử dụng chương trình này (http://bit.ly/ic4plL) để tìm strings của một file exe, chúng thường được lưu dưới dạng ASCII hoặc Unicode.
  - Mỗi khi tác giả nhắc đến từ "Unicode", đó là định dạng **"wide character string" (UTF-16LE)** theo cách cài đặt riêng của Microsoft, chứ không phải chuẩn Unicode quốc tế thông thường (như UTF-8)
  - **Unicode chuẩn (phổ biến nhất là UTF-8):** Độ dài biến đổi (từ 1 đến 4 byte). Các ký tự tiếng Anh (ASCII) cơ bản chỉ chiếm **1 byte** (Ví dụ: chữ 'A' được lưu bằng mã hex là `41`).
  - **Microsoft Wide Character (UTF-16LE):** Cố định sử dụng ít nhất **2 byte** cho mọi ký tự. Các ký tự tiếng Anh cơ bản sẽ bị chèn thêm một byte rỗng (null byte) vào sau (Ví dụ: chữ 'A' được lưu thành `41 00`).
- Cả 2 format ASCII và Unicode lưu trữ ký tự trong một trình tự mà kết thúc bằng một NULL terminator để định chỉ rằng string đã hoàn thiện. ASCII string sử dụng 1 byte/ký tự, Unicode sử dụng 2 bytes/ký tự.

→ Ví dụ về chữ “BAD” được lưu trữ trong ASCII

Figure 2-2 shows the string BAD stored as ASCII. The ASCII string is stored as the bytes 0x42, 0x41, 0x44, and 0x00, where 0x42 is the ASCII representation of a capital letter B, 0x41 represents the letter A, and so on. The 0x00 at the end is the NULL terminator.

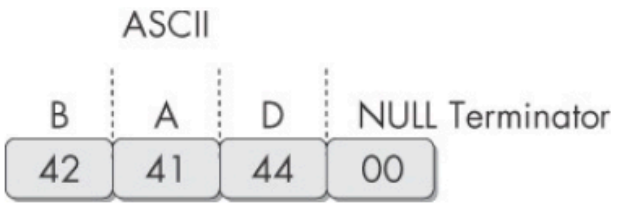

Figure 2-2. ASCII representation of the string BAD

→ Ví dụ về chữ “BAD” được lưu trữ trong Unicode

Figure 2-3 shows the string BAD stored as Unicode. The Unicode string is stored as the bytes 0x42, 0x00, 0x41, and so on. A capital B is represented by the bytes 0x42 and 0x00, and the NULL terminator is two 0x00 bytes in a row.

Figure 2-3. Unicode representation of the string BAD

Khi Strings tìm ASCII và Unicode trong một file exe, nó bỏ qua context và formatting, nên nó có thể phân tích bất cứ loại file nào, xuyên suốt cả file. (dẫn tới việc đôi khi nó sẽ xác định bytes của ký tự là strings khi nó không phải.) Strings tìm một liên kết lơn hơn hoặc bằng 3 ký tự ASCII hoặc Unicode, theo sau bởi một NULL terminator.

- Đôi khi nó sẽ xác định nhưng thứ không phải strings. Bình thường chính người dùng phải tự filter ra các strings không hợp lệ.
- Được cái là bình thường các strings không hợp lệ khá dễ phát hiện (nhìn là biết)
- Ví dụ:

```
C:>strings bp6.ex_
VP3
VW3
t$@
D$4
99.124.22.1 4
e-@
GetLayout 1
GDI32.DLL 3
SetLayout 2
M}C
Mail system DLL is invalid.!Send Mail failed to send message. 5
```

→ Có thể bỏ qua các chữ in đậm như VP3, VW3 do nó ngắn và không giống một từ có nghĩa.

→ Các Strings như “GetLayout”, “SetLayout” là các hàm được Windows graphics library sử dụng. (thường viết theo kiểu PascalCase)

→ GDI32.DLL là tên của một thư viện DLL sử dụng bởi các chương trình đồ hoạ.

→ 99.124.22.1 là một địa chỉ IP, thông báo “Mail system DLL..” là một message báo lỗi.

Điều này tiết lộ rằng file này gửi thông báo (có thể qua email), và nó phụ thuộc vào mail system DLL. Từ đó mình có thể tiếp tục phân tích với các manh mối này.

Packed and Obfuscated Malware

- Người viết malware thường sử dụng packing hoặc obfuscation để khiến file của họ khó nhận biết hoặc phân tích.
    - Những program “obfuscated” được biến đổi mã chương trình sao cho cực kỳ khó đọc và hiểu đối với con người và công cụ phân tích
    - Packed programs là một subset của obfuscated programs mà trong đó các chương trình độc hại được nén và không thể phân tích. (như dùng strings)

→ Cả 2 cách này đều giới hạn khả năng của mình để phân tích malware

Bình thường thì các chương trình hợp lệ thường luôn bao gồm nhiều strings. Malware mà packed/ obfuscated chứa rất ít strings. (nếu dùng strings mà ra ít output thì có thể file đó malicious, nhưng mình cần phân tích thêm)

- Các code mà packed và obfuscated thường bao gồm ít nhất các hàm LoadLibrary, GetProcAddress, thường được sử dụng để load và gain access cho các hàm bổ sung (lười dịch:D)

Packing Files

- Khi một chương trình packed được chạy, một chương trình wrapper nhỏ cũng được chạy để giải nén cái file packed đấy, và chạy file đã được unpacked. Khi mình phân tích file được packed thì chỉ những thử của cái chương trình wrapper được phân tích.

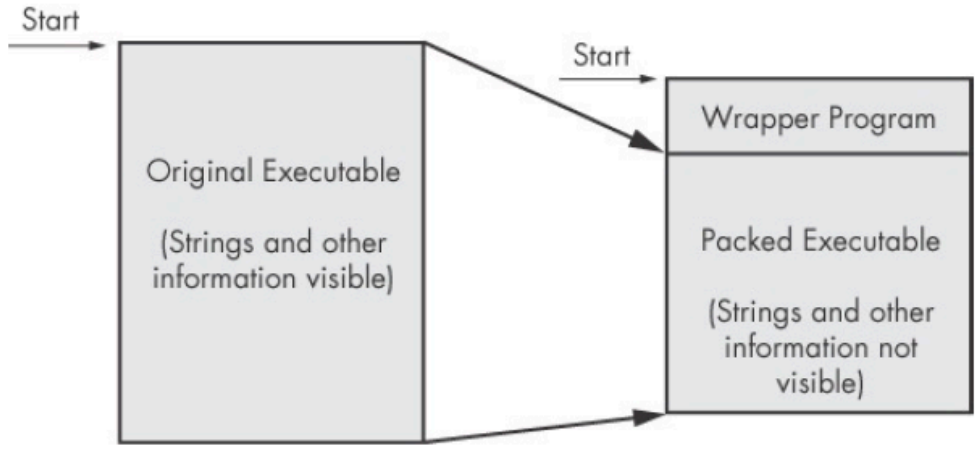

Figure 2-4. The file on the left is the original executable, with all strings, imports, and other information visible. On the right is a packed executable. All of the packed file's strings, imports, and other information are compressed and invisible to most static analysis tools.

Detecting Packers with PEiD

Một cách để phát hiện một packed file là sử dụng chương trình PEiD. PEiD được dùng để tìm ra thể loại packer hoặc compiler được sử dụng để xây dựng chương trình, khiến cho quá trình phân tích packed file đơn giản hơn.

- Tuy đã ngừng phát triển và hỗ trợ từ 2011, PEiD vẫn là tool tốt nhất cho việc phát hiện packer và compiler. Trong một vài trường hợp, nó được sử dụng để xác định packer nào được sử dụng để pack file đó.

Ví dụ 2.5 cho biết thông tin của file orig_af2.ex bởi PEiD

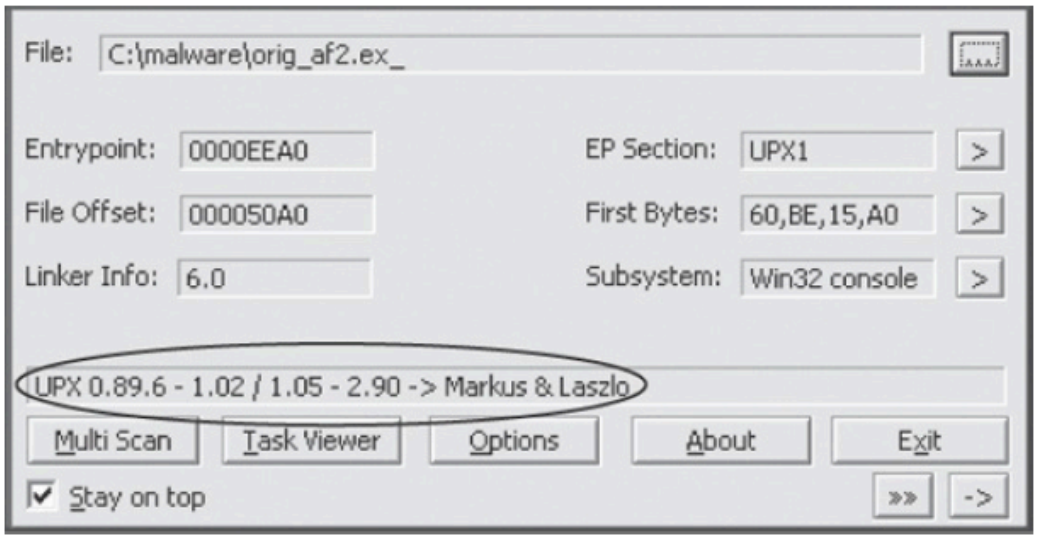

Figure 2-5. The PEiD program

- File được pack bởi UPX phiên bản 0.89.6-1.02 or 1.05-2.90

Khi một program bị packed thì tất nhiên cần unpack thì mới phân tích được. Việc unpack đc giải thích rõ hơn ở chương 19. Packing/Unpacking bằng UPX khá phổ biến và dễ dùng, mình chỉ cần tải UPX từ http://upx.sourceforge.net/ và chạy nó là được.

- Khi dùng PEiD thì HÃY TẢI BẢN MỚI NHẤT. Các bản cũ có thể có các lỗ hổng mà attacker có thể sử dụng.

Portable Executable File Format

Portable Executable (PE) là format được sử dụng bởi các file Windows executables, object code và DLL. File format này chứa thông tin cần thiết cho hệ điều hành Windows loader để quản lý các executable code được "bọc". Gần như tất cả các file được load bởi Windows có dạng PE (vẫn có thể có dạng khác).

- Các file PE bao gồm một header, chứa thông tin về code, thể loại application, các hàm thư viện cần thiết, yêu cầu về bộ nhớ. Header chứa nhiều thông tin hữu ích để mình phân tích.

Linked Libraries and Functions

- Imports là các hàm được sử dụng bởi một chương trình mà được lưu ở một chương trình khác, chẳng hạn như một thư viện mà chứa nhiều functions cho nhiều chương trình. Việc kết nối tới một executable được gọi là “linking”. Việc biết được các imports sẽ rất hưu ích.
- Các thư viện có thể được link trong lúc tĩnh/ động/ trong lúc chạy. Mình cần hiểu được cách làm thế nào thư viện được liên kết sẽ rất quan trọng, bởi vì các thông tin mình tin được ở PE Header phụ thuộc vào điều đó.

Static, Runtime, Dynamic Linking

Static Linking (nối tĩnh) là phương pháp nối với thư viện được sử dụng ít nhất, mặc dù nó phổ biến trong UNIX và Linux programs. Khi một thư viện được nối tĩnh tới một executable, tất cả code từ cái thư viện đấy được copy vào trong executable đó, khiến file đó trở nên to hơn.

- Khi phân tích thì khá là khó để phân biệt giữa code được liên kết tĩnh và code của chính executable đó.

Runtime Linking (liên kết lúc chạy) được sử dụng phổ biến nhất trong malware (ít gặp ở các chương trình thông thường), đặc biệt khi nó packed hoặc obfuscated. Executables mà sử dụng cách này chỉ liên kết tới các thư viện những lúc nó cần, và không nạp hết vào cùng lúc ngay từ khi chạy (như dynamic linking).

Một vài hàm trong Windows cho phép mình link sao cho không được hiện ở trong Header. Trong đó có 2 hàm phổ biến nhất:

- LoadLibrary, GetProcAddress.
- LdrGetProcAddress và LdrLoadDll cũng được sử dụng.

LoadLibrary, GetProcAddress, cho phép chương trình truy cấp bất cừ hàm nào ở thư viện nào trong hệ thống, nghĩa là khi mấy hàm này được sử dụng thì mình sẽ không biết hàm nào được liên kết tĩnh bởi chương trình đó.

Trong tất cả các cách link thì dynamic linking(liên kết động) phổ biến nhất và thú vị nhất đối với người phân tích malware. Khi các thư viện được nối động, hệ điều

hành của máy chủ sẽ tìm các thư viện cần thiết khi mà chương trình được chạy. Khi mà chương trình gọi hàm, thì hàm đó sẽ chạy ở trong thư viện.

→ PE header lưu các thông tin về tất cả các thư viện mà sẽ được load và tất cả các hàm được sử dụng bởi chương trình. Nếu biết được điều này thì mình có thể đoán chương trình sẽ làm gì.

Exploring Dynamically Linked Functions with Dependency Walker

The Dependency Walker sẽ liệt kê chỉ những hàm được nối động ở trong một executable.

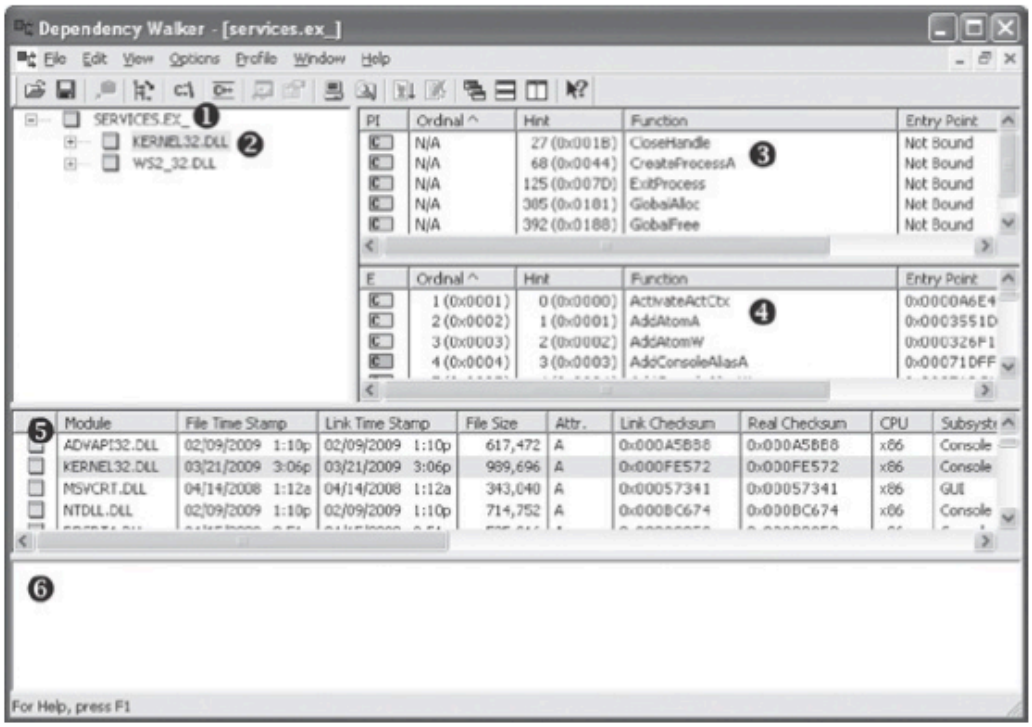

Figure 2-6. The Dependency Walker program

- Phân tích của Dependency Walker cho một file SERVICES.EX_. Phần số 2 ở bên trái hiển thị cái chương trình và các DLL được import, bao gồm KERNEL32.DLL, WS2_32.DLL.
- Ở ô có ghi số 3, là các hàm được import. Có thể thấy có một hàm là “CreateProcessA” thì mình có thể đoán được là chương trình sẽ tạo một chương trình con khác trong lúc chạy.

- Ở chỗ số 4 là các hàm trong KERNEL32.DLL mà có thể được import-thông tin này k hữu dụng lắm.
- Ở trong cả ô số 3 và 4 đều có cột “Ordinal”(số thứ tự), nghĩa là file có thể import các hàm theo số thứ tự thay vì tên. Và nếu làm như vậy thì tên của hàm sẽ không bao giờ xuất hiện ở trong executable. (khiến mình khó phân tích hơn). Khi một hàm được import bằng ordinal, thì mình có thể tìm ra đó là hàm nào bằng cách nhìn số tương ứng với nó ở ô số 4

→ Kiểu ô số 3 là các hàm import, nếu mà nó không ghi rõ tên mà chỉ có ordinal, thì có thể lấy số đó xong tra trong ô số 4.

- Ô số 5 và 6 chưuas các thông tin bổ sung về phiên bản của DLL nếu chương trình được chạy, và các lỗi nào xuất hiện.

DLL của một chương trình sẽ cho mình biết nhiều về tính năng của chương trình đó. Đây là bảng các DLL thường được sử dụng và nó nói gì về chương trình của mình.

Table 2-1. Common DLLs

DLL	Description
Kernel32.dll	This is a very common DLL that contains core functionality, such as access and manipulation of memory, files, and hardware.
Advapi32.dll	This DLL provides access to advanced core Windows components such as the Service Manager and Registry.
User32.dll	This DLL contains all the user-interface components, such as buttons, scroll bars, and components for controlling and responding to user actions.
Gdi32.dll	This DLL contains functions for displaying and manipulating graphics.
Ntdll.dll	This DLL is the interface to the Windows kernel. Executables generally do not import this file directly, although it is always imported indirectly by Kernel32.dll. If an executable imports this file, it means that the author intended to use functionality not normally available to Windows programs. Some tasks, such as hiding functionality or manipulating processes, will use this interface.
WSock32.dll and Ws2_32.dll	These are networking DLLs. A program that accesses either of these most likely connects to a network or performs network-related tasks.
Wininet.dll	This DLL contains higher-level networking functions that implement protocols such as FTP, HTTP, and NTP.

Một số chuẩn trong việ đặt tên hàm của Windows

FUNCTION NAMING CONVENTIONS

When evaluating unfamiliar Windows functions, a few naming conventions are worth noting because they come up often and might confuse you if you don't recognize them. For example, you will often encounter function names with an `Ex` suffix, such as `CreateWindowEx`. When Microsoft updates a function and the new function is incompatible with the old one, Microsoft continues to support the old function. The new function is given the same name as the old function, with an added `Ex` suffix. Functions that have been significantly updated twice have two `Ex` suffixes in their names.

Many functions that take strings as parameters include an A or a W at the end of their names, such as `CreateDirectoryW`. This letter does not appear in the documentation for the function; it simply indicates that the function accepts a string parameter and that there are two different versions of the function: one for ASCII strings and one for wide character strings. Remember to drop the trailing `A` or `W` when searching for the function in the Microsoft documentation.

Imported Functions

PE HEader bao gồm thông tin về các hàm cụ thể được sử dụng bởi một file executable. Tên của các hàm này cho mình biết được là file sẽ làm gì. Microsoft thường document các hàm này trong thư viện Microsoft Developer Network (MSDN) (các hàm thường được malware sử dụng ở trong Appendix A)

Exported Functions

Giống với Imports, DLLs và EXEs export các hàm để giao tiếp với các chương trình/ code khác. Thường thì một DLL sẽ áp dụng một hoặc nhiều hàm và export chúng để một executable sử dụng.

File PE chứa thông tin về các hàm nào mà một file export. Do vậy thường các hàm export chứa nhiều nhất trong các thư viện DLL, để được sử dụng bởi các EXEs. EXEs không được thiết kế để cung cấp tính năng cho các EXEs khác. (các hàm export thường khá hiếm) Nếu mình tìm ra được hàm export thì nó sẽ có các thông tin hữu ích.

Bình thường thì người lập trình sẽ đặt tên cho hàm một cách dễ hiểu (bao gồm tính năng của nó). Một chuẩn là đặt tên giống trong Microsoft documentation. Tất nhiên là hàm vẫn có thể được đặt thành bất cứ tên gì, và nếu malware sử dụng tên của hàm thì thường sẽ che giấu tên, sử dụng các tên sai lệch hoặc không liên quan hoàn toàn:)))

> For example, in order to run a program as a service, you must first define a ServiceMain function. The presence of an exported function called ServiceMain tells you that the malware runs as part of a service

The PE File Headers and Sections

PE Header chứa nhiều thông tin hơn, không chỉ dừng lại ở các hàm mà nó import. Nó chứa một header và theo sau bởi một chuỗi các sections, mỗi section chứa các thông tin thú vị.. Phần Header sẽ chứa metadata của chính file đó. Đây là các sections phổ biến và thú vị:

- .text: Chứa các instructions mà CPU chạy. Tất cả các sections khác chứa data và thông tin bổ sung. Thường thì đây là section duy nhất mà có thể execute, và

sẽ là section duy nhất chứa code.

- .rdata: Chứa các thông tin import và export, chính là các thông tin trong PEview và Dependency Walker. Phần này cũng có thể chứa các read-only data sử dụng ở trong file. Thỉnh thoảng một file chứa .idata hoặc .edata section sẽ chứa thông tin import và export tương ứng
- .data: Chứa data toàn cục (global) của chương trình, có thể được truy cập từ bất cứ đâu. Data cục bộ (Local) không chứa trong đây hay nơi khác ở file PE (Chương 7)
- .rsrc: Chứa cáctaif nguyên được sử dụng bởi file exe mà không được coi là một phần của file exe, giống kiểu icons, images, menus, và strings. Strings có thể đc lưu ở main hoặc ở section này, nhưng thường lưu ở section này.

Tên các sections thường cónsistent giữa một compiler, nhưng có thể khác nhau giữa các compiler. Tên Section có thể được che giấu để việc phân tích khó hơn. (may là thường các tên default xuất hiện nhiều nhất)

Table 2-4. Sections of a PE File for a Windows Executable

Executable	Description
`.text`	Contains the executable code
`.rdata`	Holds read-only data that is globally accessible within the program
`.data`	Stores global data accessed throughout the program
`.idata`	Sometimes present and stores the import function information; if this section is not present, the import function information is stored in the `.rdata` section
`.edata`	Sometimes present and stores the export function information; if this section is not present, the export function information is stored in the `.rdata` section
`.pdata`	Present only in 64-bit executables and stores exception-handling information
`.rsrc`	Stores resources needed by the executable
`.reloc`	Contains information for relocation of library files

Examining PE Files with PEview

đoạn này sử dụng PEview để phân tích một vài malware

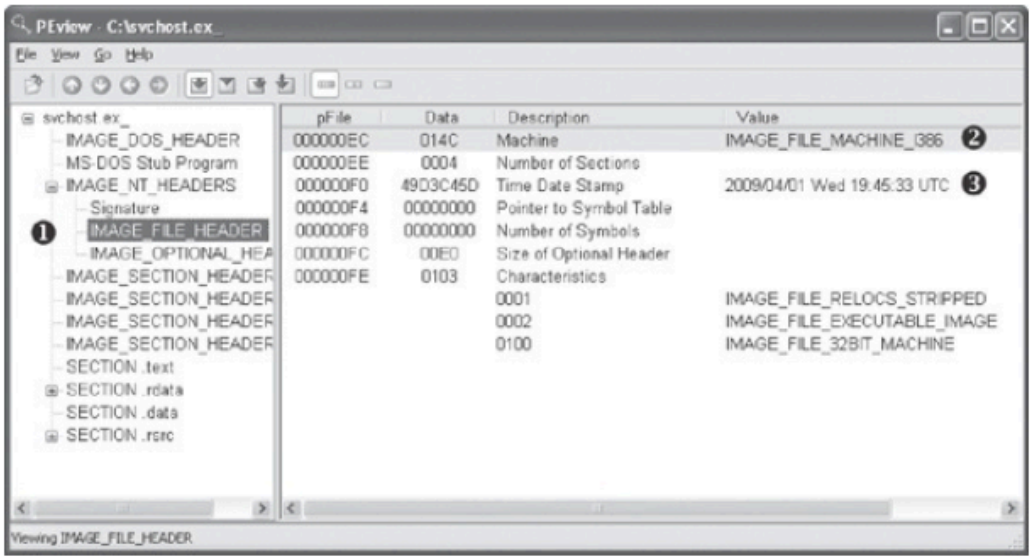

Figure 2-7. Viewing the `IMAGE_FILE_HEADER` in the PEview program

phần này lười dịch quá

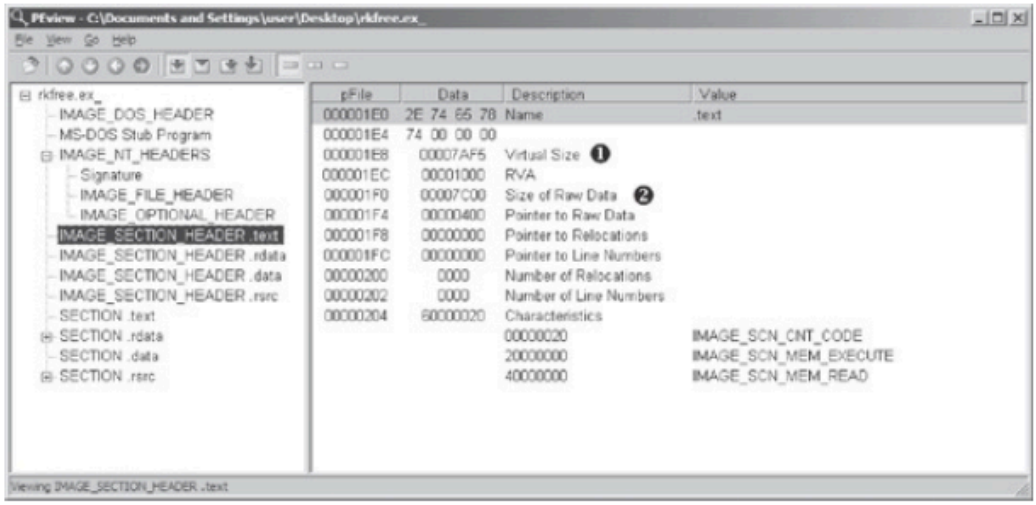

Figure 2-8. Viewing the `IMAGE_SECTION_HEADER .text` section in the PEview program

Điều cần lưu ý là, sẽ có một vài file mà có Virtual Size (dung lượng cần trong quá trình chạy), và Size of Raw Data (dung lượng file chiếm trên đĩa). Nếu mà Virtual Size to hơn Size of Raw Data, thì có nghĩa đây là packed file, đặc biệt là khi section .text nó lớn hơn ở trong bộ nhớ hơn trên đĩa.

Nó sẽ nhìn như này:

Table 2-6. Section Information for PackedProgram.exe

Name	Virtual size	Size of raw data
.text	A000	0000
.data	3000	0000
.rdata	4000	0000
.rsrc	19000	3400
Dijfpds	20000	0000
.sdfuok	34000	3313F
Kijijl	1000	0200

→ Ngoài ra trong phần section này có vài tên khá lạ, và size of raw data bằng 0, nghĩa là khi chạy sẽ có một wrapper giúp giải nén thành file thực tế vào phần .text

Viewing the Resource Section with Resource Hacker

Phần section duy nhất mà mình có thể hiểu ở thời điểm hiện tại là Resource Section. Mình có thể dùng Resource Hacker, mình sẽ thấy những strings, icons, menus. Menus hiển thị sẽ giống y hệt như menus mà chương trình dùng.

Đây là Resource Hacker với “Calculator”

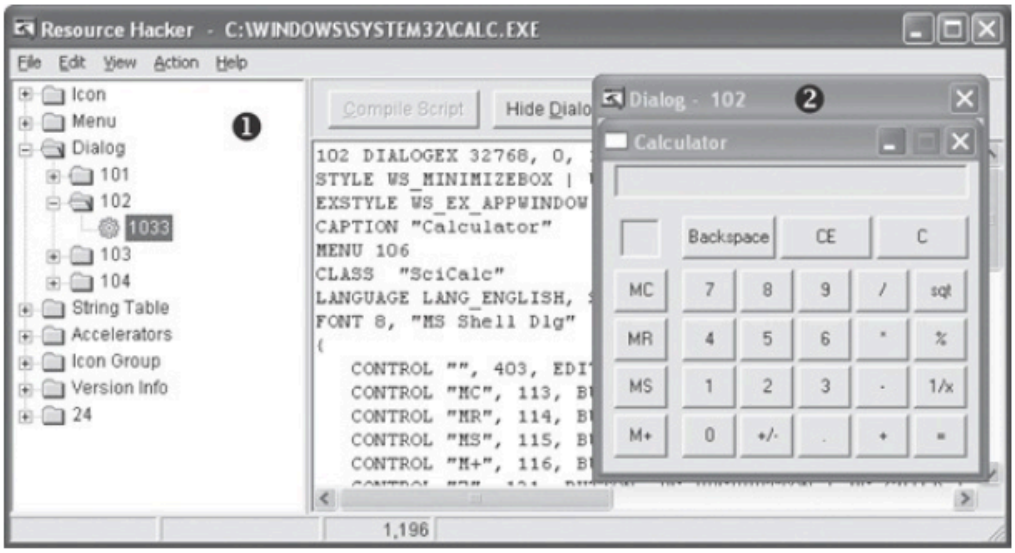

Figure 2-9. The Resource Hacker tool display for calc.exe

- Phần panel 1 chứa tất cả tài nguyên mà exe sẽ sử dụng, mỗi folder chứa một loại tài nguyên.
- Các phần mà mình cần sẽ chứa:
  - Icon: liệt kê hình ảnh mà được cho xem khi file đó được liệt kê
  - Menu: chứa các menus mà xuất hiện ở các cửa sổ, chẳng hạn File, Edit, Views menus. Vậy nên nó chứa tên của tất cả các menus, và ký tự của từng cái. (khá hữu dụng)
  - Dialogues: chứa dialog menus của chương trình, số 2 cho thấy người dùng sẽ thấy gì khi chạy calc.exe
  - Strings: chứa strings
  - Version Info: chứa version number, tên công ty và copyright statement
- Malware và thỉnh thoảng các phân mềm bình thường sẽ embed các chương trình và driver vào đây, trước khi nó chạy nó sẽ trích xuất exe hoặc driver ra. Resource Hacker cho mình trích xuất các file này để phân tích.

Using Other PE Fiole Tools

Có một vài tool khác như: PEBrowse Professional, PE Explorer

Tổng lại quan PE Header chứa thông tin gì?

Field	Information revealed
Imports	Functions from other libraries that are used by the malware
Exports	Functions in the malware that are meant to be called by other programs or libraries
Time Date Stamp	Time when the program was compiled
Sections	Names of sections in the file and their sizes on disk and in memory
Subsystem	Indicates whether the program is a command-line or GUI application
Resources	Strings, icons, menus, and other information included in the file